SnapPark Dissertation — Detailed Explanation

SnapPark Dissertation — Detailed Chapter-by-Chapter Explanation
Front matter — the first few pages
- Title page
- Abstract
- Declaration
- Acknowledgments
- Contents, List of Figures, List of Tables, Listings

Chapter 1 — Introduction (in detail)
- Opening paragraphs (before §1.1)
- §1.1 — Aim of the project
- §1.2 — Project objectives
- §1.3 — Report structure

Chapter 2 — Literature Review (in detail)
- §2.1 — Architectural Paradigms
  - §2.1.1 — Monolithic Architecture
  - §2.1.2 — Service-Oriented Architecture (SOA)
  - §2.1.3 — Microservices Architecture
  - §2.1.4 — Choice of Architecture
- §2.2 — Cloud Computing and the Cloud Continuum
- §2.3 — Challenges in Microservice-Based Designs
  - §2.3.1 — Microservice Identification and Domain-Driven Design
  - §2.3.2 — Data Management
  - §2.3.3 — API Gateway
  - §2.3.4 — Service Discovery
  - §2.3.5 — Communication: Sync vs Async
  - §2.3.6 — Containerisation and Orchestration
- §2.4 — AI and LLMs in Image Analysis
- §2.5 — Similar Projects
- §2.6 — Implementation Technologies

Chapter 3 — System Description and Requirements (in detail)
- §3.1 — Bird's-Eye View of the System
- §3.2 — Functional Requirements (FR1–FR8)
- §3.3 — Non-Functional Requirements (NFR1–NFR8)

Chapter 4 — Project Management (in detail)
§4.1 — Risk Management
§4.2 — Software Development Process
Chapters 5–8 — The Four Increments
Chapter 5 — Increment 1: API Gateway + Authentication
§5.1 — Analysis
§5.2 — Design
§5.3 — Implementation
§5.4 — Testing
§5.5 — Evaluation
Chapter 6 — Increment 2: Violation Analysis with Gemini
§6.1 — Analysis
§6.2 — Design
§6.3 — Implementation
§6.4 — Testing
§6.5 — Evaluation
Chapter 7 — Increment 3: Event-Driven Notifications
§7.1 — Analysis
§7.2 — Design
§7.3 — Implementation
§7.4 — Testing
§7.5 — Evaluation
Chapter 8 — Increment 4: Deployment and Orchestration
§8.1 — Analysis
§8.2 — Design
§8.3 — Implementation
§8.4 — Testing
§8.5 — Evaluation
Chapter 9 — Evaluation (in detail)
§9.1 — Evaluation of Objectives
§9.2 — User Interface
Chapter 10 — Conclusion (in detail)
Opening
§10.1 — Project Challenges
§10.2 — Future Work
§10.3 — Final Words
References
How the chapters connect
Things to remember for the viva

SnapPark Dissertation — Detailed Chapter-by-Chapter Explanation

Purpose. This document takes the dissertation (`DISSERTATION.pdf`) and walks through it, **paragraph by paragraph**, explaining in plain English:

- what each section is saying,
- why that section exists,
- how it connects to your actual code,
- and what an examiner is likely to look for there.

Read it side-by-side with the dissertation. Every numbered heading here matches a numbered section there.

Verification note. Before writing this guide, I re-verified the technical claims in the dissertation against your actual code. Three small factual errors were found and corrected in the dissertation:

1. Ports in §8.2 — notification service is on port 3004, not 3003. Fixed.
2. Case lifecycle states in §6.3.3 — the real states are `pending, completed, reported_to_authority, resolved, cancelled, expired` (not `analysed`). Fixed.
3. §6.3.3 previously stated the `images` and `analysis_results` tables are created on startup; in reality only the `cases` table is fully populated by the running service, while the auxiliary tables are documented in the schema and populated as the feature surface expands. Softened to match reality.

Everything else the dissertation claims (file line counts, bcrypt cost, JWT expiries, blur threshold, Laplacian kernel, queue names, channel classes, rate limit, Kubernetes HPA, OpenAPI spec) was verified as correct against the code.

Front matter — the first few pages

Title page

What it says. “SnapPark: An Intelligent Microservices-Based Parking Violation Detection System”, submitted in partial fulfilment of a BSc Computer Science at CITY College (University of York Europe Campus), May 2026, supervised by Dr. Simeon Veloudis.

Why it’s there. Every dissertation must identify author, degree, institution, date and supervisor on page one — examiners, external markers and the library all rely on it.

Abstract

What it says. A single paragraph that summarises the whole dissertation: the problem (illegal parking is hard to enforce), the proposed solution (a microservices back-end where citizens upload photos and an LLM produces a reasoned verdict), the architectural ingredients (Cloud-Continuum, Database-per-Service, event-driven messaging), and the structure of the report.

Why it’s there. The abstract is the first thing a marker reads and often the only thing a stranger will read. It has to tell the whole story in ~250 words. Think of it as the “elevator pitch” version of the dissertation.

Declaration

What it says. A standard academic integrity statement: all quoted work is referenced, the work was done without unauthorised assistance, and the dissertation may be reused as a teaching example.

Why it’s there. Required by the university. It is the equivalent of a signed contract stating you have not plagiarised. You sign and date it before submission.

Acknowledgments

What it says. Thanks to the supervisor, the department and your family. Kept short and professional.

Why it’s there. Convention. It costs nothing and is expected.

Contents, List of Figures, List of Tables, Listings

What they are. Navigation aids. The contents lists chapters with page numbers; the three specialised lists enumerate every figure, every table and every code listing, with captions.

Why they're there. An examiner who wants to revisit Figure 6.2 (the analysis sequence diagram) should find it in under ten seconds. These lists make that possible.

Chapter 1 — Introduction (in detail)

Opening paragraphs (before §1.1)

The introduction opens by describing **why illegal parking matters**. It is not only an inconvenience — blocked pedestrian crossings endanger people with reduced mobility, blocked fire lanes delay emergency services, and the cumulative effect of ignored parking rules degrades the livability of a city. This framing tells the examiner that the problem is real and socially relevant, not a toy.

The second paragraph identifies three independent technology trends that have matured at the same time and make SnapPark feasible *now*: - **Ubiquitous smartphone cameras** — every citizen carries a high-resolution camera. - **Commercial vision-capable LLMs** (Gemini, GPT-4V, Claude) — you can now classify images with natural-language reasoning without training your own neural network. - **Microservices architecture** — the software engineering community has settled on a pattern for distributed, independently scalable systems.

The third paragraph describes the runtime flow: *client → API Gateway → Authentication Service → Violation Analysis Service → Gemini → persistence → Notification Service (asynchronously)*. This is effectively the executive summary of the rest of the dissertation.

The fourth paragraph states what the dissertation itself sets out to do: document the literature, the design decisions, the alternatives that were rejected, the implementation of each increment, the testing strategy, and an honest evaluation.

§1.1 — Aim of the project

What it says. In one paragraph: the aim is to design, implement and evaluate a microservices back-end that lets citizens report suspected parking violations with one or more images, and that uses an LLM to produce a structured, reasoned verdict — deployable both under Docker Compose and under Kubernetes.

What it deliberately excludes. The second paragraph makes explicit that SnapPark does **not** aim to replace a human traffic warden. This is a defence against a common viva question: “but is this good enough to be used in production?”. The answer is “that is not the aim; the aim is to demonstrate that the architecture is viable.”

Examiner angle. Markers look for a single, unambiguous statement of aim. Vague aims (“build a cool system with AI”) lose marks; specific aims (“design, implement and evaluate a microservices back-end that…”) earn them.

§1.2 — Project objectives

Six objectives are listed, each one a *qualitative property* of the final system:

1. **Scalability** — parts of the system that experience more load (the Violation Analysis Service under a surge of reports) must scale independently of the others. *Connection to code:* realised through independent Docker images and the Kubernetes HorizontalPodAutoscaler on the analysis service.
2. **High availability** — a crashing Notification Service must not stop analyses, and a crashing Authentication Service must not stop users whose tokens are still valid. *Connection to code:* the RabbitMQ broker absorbs notification-side failures; health-checks let Kubernetes restart failing pods.
3. **Security** — every request authenticated, no plain-text passwords, short-lived access tokens, refresh-token rotation. *Connection to code:* bcrypt at cost 10, 15-minute JWTs, refresh-token rotation in `authentication-service`.
4. **Auditability** — every significant state change leaves a tamper-resistant trace. *Connection to code:* `case.*` events on the RabbitMQ `snappark` exchange; an append-only event store is designed in `databases/schema.md` but not yet wired up.
5. **Extensibility** — a new notification channel, a new consumer or a new microservice must not require changing existing code. *Connection to code:* the `BaseChannel` class hierarchy in `services/notification-service/src/channels/`.
6. **Maintainability** — each service small enough for one developer to hold in their head, clearly documented, independently testable. *Connection to code:* each service is under ~1,500 LOC, has its own README and its own test folder.

Why these matter later. Chapter 9 will come back and check each objective; §1.2 is the scoring sheet the examiner uses.

§1.3 — Report structure

A map of the remaining chapters. Routine but expected — without it, the reader doesn't know what is coming.

Chapter 2 — Literature Review (in detail)

This is the longest theory chapter. Its job is to **prove you understand the field before you touched code**. The chapter opens by saying the architectural paradigm is the substrate on which every later choice rests — i.e. if you get the architecture wrong, nothing else can save the project.

§2.1 — Architectural Paradigms

Three paradigms are compared.

§2.1.1 — Monolithic Architecture

A **monolith** is one deployable unit: the UI, the business logic, the data layer and the infrastructure are all in one codebase and one process. One database.

Advantages. Single deploy, easy local dev, easy integration tests, fast in-process calls.

Disadvantages (the tables spell these out): - Poor maintainability — a one-line change means rebuilding and redeploying everything. - Tight coupling — modules share a database schema; changes in one module ripple into another. - Poor scalability — you can only scale the whole app, not the hot spot. - Poor availability — a bug in one module can take down the whole process. - Technology lock-in — whole app is one language.

Why this matters to SnapPark. These weaknesses directly violate your objectives 1 (scalability) and 2 (availability). So monolith is ruled out.

§2.1.2 — Service-Oriented Architecture (SOA)

SOA emerged in the mid-2000s. It breaks a monolith into reusable services that talk through a central **Enterprise Service Bus (ESB)**. Messages are typically XML over SOAP.

Advantages. Reusable services, independent teams, a natural place for policy enforcement (the ESB).

Disadvantages. The ESB is itself a bottleneck and a single point of failure. Services are coarse-grained. SOAP/XML is heavyweight. Operating an ESB is non-trivial.

Why this matters to SnapPark. The ESB would be a single point of failure (violates availability objective), and SOAP/XML is painful when every request carries a multi-megabyte image. So SOA is ruled out.

§2.1.3 — Microservices Architecture

Microservices are “SOA done right”: no central ESB, “smart endpoints and dumb pipes”. Each service is small, owns its data, has its own database, and talks over lightweight protocols (HTTP/JSON, gRPC, or async messages via a broker).

Advantages. - **Independent scalability** — each service scales on its own curve. - **Fault isolation** — a failing service stays contained. - **Technology diversity** — polyglot (Node, Python, Go, Java, etc.). - **Independent deployability** — teams release on their own cadence. - **Organisational alignment (Conway’s law)** — small services map to small teams.

Disadvantages. - **Operational complexity** — dozens of services to monitor, log, trace. - **Network overhead** — in-process calls become network calls with new failure modes. - **Data consistency** — no ACID transactions across services; must use Saga (§2.3.2). - **Infrastructure cost** — each service is its own container.

§2.1.4 — Choice of Architecture

The dissertation now makes the decision explicit. The SnapPark requirements *force* microservices: independent scaling is explicit, availability demands fault isolation, extensibility demands pub/sub. Monolith is out (can’t scale independently). SOA is out (ESB is SPOF, SOAP is heavy). Microservices wins.

Examiner angle. Markers want to see **justified** choices. “I chose microservices because they are cool” fails. “I chose microservices because my scalability and extensibility objectives are incompatible with a monolith, and the SOA alternative reintroduces the single-point-of-failure I am trying to avoid” passes.

§2.2 — Cloud Computing and the Cloud Continuum

Cloud Computing is compute/storage/networking as a service over the internet (pay-as-you-go instead of buying servers). AWS, Azure, GCP are the big three.

The **Cloud Continuum** is an extension of the idea: computation is no longer anchored in a few data centres, but distributed along a continuum:

```
far edge (phone) → near edge (regional datacentre) → cloud
(hyperscaler)
```

Two reasons to care: 1. **Latency** — a phone analysing a frame for a driving-assistance system cannot afford a 200-ms round trip to a distant cloud. 2. **Bandwidth and privacy** — processing data close to its source reduces both.

How SnapPark fits. Image validation (resolution, brightness, blur) could run on the phone or on the near edge. The Gemini call, which is heavy, belongs in the cloud. Microservices are the right unit of deployment for this because each service can be placed wherever it fits best.

§2.3 — Challenges in Microservice-Based Designs

Microservices solve some problems and create new ones. Six sub-sections discuss the new problems.

§2.3.1 — Microservice Identification and Domain-Driven Design

A bad microservice split produces a **distributed monolith** — services that cannot be deployed independently, *and* all the operational pain of a distributed system. Bad of both worlds.

Domain-Driven Design (DDD), from Eric Evans, helps. Its core idea is the **bounded context** — a region of the domain with its own consistent language. Services are drawn around bounded contexts, not around technical layers.

For SnapPark, three bounded contexts produce three microservices: - *User, Credential, RefreshToken* → Authentication Service. - *Case, Image, AnalysisResult* → Violation Analysis Service. - *Notification, Channel, Preference* → Notification Service.

§2.3.2 — Data Management

Microservices follow **Database per Service**: each service owns its data; nobody else touches it directly. You want something from another service? Call its API or consume its events.

Consequence. Cross-service queries are no longer a JOIN; ACID transactions across services are gone. Three patterns address the gap:

- **CQRS (Command Query Responsibility Segregation).** Split the write model (validates rules, emits events) from the read model (populated from events, shaped for queries). Useful when reads far outnumber writes.
- **Event Sourcing.** Don't store only current state — store the *history of events* that produced it. Replay to reconstruct. This gives you a natural immutable audit log, which maps directly to Objective 4 (Auditability). SnapPark emits events; the append-only event store is designed in the schema and is the biggest future-work item.

- **Saga Pattern.** A cross-service "transaction" becomes a sequence of local transactions, each emitting an event. If one fails, compensating transactions undo the preceding ones. Two flavours: **choreographed** (services react to each other's events) and **orchestrated** (a central controller drives).

§2.3.3 — API Gateway

Without a gateway, clients must know the address of every microservice and handle cross-cutting concerns (auth, rate limits) themselves. The **API Gateway pattern** introduces a single public front door that handles routing, authentication, rate limiting, transformation and observability. The gateway is **infrastructural** — it has no business logic and no business database.

SnapPark uses a hand-rolled Express.js gateway rather than an off-the-shelf product (Kong, AWS API Gateway) to keep the implementation transparent and small.

§2.3.4 — Service Discovery

In a dynamic environment (services scale up/down, IPs change), hard-coded addresses are a liability. **Service Discovery** lets services register on startup and lets clients look up current locations.

Two flavours: - **Client-side** (client asks the registry). - **Server-side** (a load balancer asks on your behalf).

Kubernetes implements server-side: DNS names like `authentication-service.snappark.svc` resolve to live pods behind a load balancer. SnapPark relies on this rather than running its own Consul/Eureka.

§2.3.5 — Communication: Sync vs Async

- **Synchronous** (HTTP/JSON, gRPC) — caller waits for the response. Easy to reason about. Downsides: **temporal coupling** (both must be live at the same time) and **cascading failure** (one slow callee slows the whole chain).
- **Asynchronous** (pub/sub through a broker) — publisher fires-and-forgets, broker stores the message, consumers process when ready. Decouples producer from consumer in time. Canonical foundation for event-driven architectures. Directly enables extensibility: a new consumer only has to subscribe.

SnapPark uses **both**: the citizen-facing upload → analyse → verdict flow is synchronous (user is waiting); notifications and audit are asynchronous (off the critical path).

§2.3.6 — Containerisation and Orchestration

Docker packages an app with its dependencies into an immutable image that runs identically everywhere. **Docker Compose** brings up a multi-container app on one host — perfect for dev and small demos.

Kubernetes is for production. It schedules containers across a fleet, restarts crashed containers, scales deployments, routes traffic, and manages configs, secrets, ingress and autoscaling. SnapPark ships Kubernetes manifests for all components, so the same containers run in both environments.

§2.4 — AI and LLMs in Image Analysis

Old approach. Train a Convolutional Neural Network on hundreds of thousands of labelled parking images. Cost: a curated dataset, GPUs, and continuous retraining as data drifts.

New approach. Use a vision-capable LLM (Gemini, GPT-4V, Claude) that is already pre-trained on internet-scale multimodal data. Prompt it in natural language, receive a structured response. Dramatically lower development and operational cost; no dataset to curate; the output already includes the natural-language explanation the citizen expects.

Trade-offs (discussed fully in §6 and §10). - LLM can hallucinate. - Latency higher than a small local classifier. - Per-request API cost. - Privacy: user images are sent to a third party.

For a dissertation-scale project, the trade-offs are acceptable. Crucially, the design confines Gemini behind **a single module** (`services/violation-analysis-service/src/gemini.js`), so it can be swapped for a self-hosted model without touching the rest of the system.

§2.5 — Similar Projects

- **FixMyStreet** (UK, mySociety) — citizens report potholes, graffiti, dumping. Pure reporting, no automated analysis.
- **SeeClickFix** (US) — similar.
- **ANPR systems** — fixed cameras, plate recognition only, no scene reasoning.

None combine citizen reporting with AI-based visual reasoning the way SnapPark does.

The two reference dissertations (Berisha, Osmani) are acknowledged as methodological inspiration.

§2.6 — Implementation Technologies

Four server-side stacks were considered.

- **Node.js + Express.js** — single-threaded non-blocking event loop, great for I/O, huge ecosystem, small containers, fast startup. **Chosen.**
- **Python + Flask/FastAPI** — great for data science, but SnapPark is primarily an integration-and-architecture problem.
- **Spring Boot (Java)** — enterprise gold standard, but verbose, heavy, slow startup.
- **Go** — great concurrency, small binaries, but a smaller ecosystem for Gemini/notifications SDKs.

Why Node.js won. I/O-bound workload, Gemini SDK supported, Express learning curve low, homogeneous stack simplifies single-developer reasoning, sub-second container startup.

Chapter 3 — System Description and Requirements (in detail)

The bird's-eye view and the formal requirements. Markers use these as the yardstick for evaluation in Chapter 9.

§3.1 — Bird's-Eye View of the System

Five layers:

1. **Client layer** — web or mobile app. *Not implemented in this dissertation*; all interactions via curl/Postman. Any client that speaks the documented REST API can plug in without changes.
2. **Access layer** — the API Gateway (Node.js/Express). Rate limiting, input validation, auth delegation, routing. No business logic.
3. **Core service layer** — three business services: Authentication (also security back-end), Violation Analysis (validates images, calls Gemini, persists cases), Notification (fans out notifications across channels).
4. **Messaging layer** — RabbitMQ broker with a topic exchange called `snappark`. Publishes `case.created`, `case.reported`, `case.resolved` events.
5. **Data layer** — three PostgreSQL 15 databases: `snappark_auth`, `snappark_case`, `snappark_notifications`. A fourth database (`snappark_audit`) is designed but not populated by the current implementation.

§3.2 — Functional Requirements (FR1–FR8)

These are *what the system does*, written as user-visible capabilities.

- **FR1 — Image Upload.** User can upload one or more images (multipart or base64 JSON) as evidence.
- **FR2 — User Authentication.** Nothing processed until the user has a valid bearer token. Registration and login are dedicated endpoints.
- **FR3 — Image Validation.** Every image is validated for type, size, resolution, brightness, sharpness. Failures → HTTP 422 with a machine-readable reason.
- **FR4 — Case Record Retention.** Each submission creates a persisted *case* with metadata and analysis result.
- **FR5 — Analysis Notification.** The user is notified once analysis is done, through every channel they have enabled.
- **FR6 — Report Cancellation.** User can cancel up until the analysis is confirmed and forwarded to authorities.

- **FR7 — Image Cleanup.** Uploaded images that sit unprocessed past a configurable threshold are auto-discarded.
- **FR8 — Multiple Image Submission.** Up to five images per report; all analysed together.

§3.3 — Non-Functional Requirements (NFR1–NFR8)

How well the system does what it does.

- **NFR1 — Scalability.** Horizontal scaling per service without redeploying the rest. Target: 99.5% availability under normal load; graceful degradation under peak.
- **NFR2 — High Availability.** Failure of any component doesn't propagate. Broker decouples non-critical paths. `/health` endpoints for liveness probes.
- **NFR3 — Security.** Valid, unexpired access token on every core-service request. bcrypt ≥ cost 10. Access tokens 15 min. Refresh tokens rotated on every use.
- **NFR4 — Performance.** Synchronous path under 5 seconds for a single image under normal load. Async paths have no fixed deadline.
- **NFR5 — Maintainability.** Each service independently deployable. No shared library across services (except infrastructure — e.g. logging). Per-service README and tests.
- **NFR6 — Auditability.** Every relevant state change emitted as an event. Append-only event store designed; implementation is future work.
- **NFR7 — Extensibility.** New notification channel = new `BaseChannel` subclass + env var. Nothing else to change.
- **NFR8 — Data Integrity.** No service reads/writes another service's database directly. Cross-service data flows via APIs or events.

Why this structure. Each NFR maps back to an objective in §1.2 and forward to a table cell in §9.1. Traceability.

Chapter 4 — Project Management (in detail)

How the project was run, as a project.

§4.1 — Risk Management

Six risks, quantified along *probability* and *impact*, each with a mitigation. Summarised in Table 4.1.

- **R1 — Gemini API change or outage (Prob=Medium, Imp=High).** Mitigation: wrap every Gemini call behind a single interface (`services/violation-analysis-service/src/gemini.js`) so the whole system can switch provider with one file change.
- **R2 — Under-estimated microservices operational overhead (Prob=High, Imp=Medium).** Mitigation: defer Kubernetes to the final increment; use Docker Compose until then.
- **R3 — Scope creep (Prob=High, Imp=Medium).** Mitigation: fix the service count at four (Gateway, Auth, Analysis, Notification). Everything else → future work.
- **R4 — Loss of work (Prob=Low, Imp=High).** Mitigation: Git + GitHub remote, commit at least daily.
- **R5 — RabbitMQ library incompatibility (Prob=Low, Imp=Medium).** Mitigation: pin `amqplib` version; reconnection with exponential back-off.
- **R6 — Supervisor unavailability (Prob=Low, Imp=Low).** Mitigation: bi-weekly meetings booked in advance.

Why include this chapter. Examiners look for **project maturity** — evidence that you thought about what could go wrong before it did. Without this chapter, the dissertation would look like code with no surrounding plan.

§4.2 — Software Development Process

Four processes were considered:

- **Waterfall.** Linear: Requirements → Design → Implementation → Test → Maintenance. Each phase finishes before the next starts. *Problem:* brittle under changing requirements, which is the norm for a student project.
- **Incremental.** Ship the system in slices; each slice is a usable subset. Within a slice, do a mini-waterfall. Errors in earlier slices can be corrected later. **Selected** for SnapPark — it maps naturally to "one microservice per increment".

- **Iterative.** Same slice, refined repeatedly. Less additive.
- **Agile/Scrum.** Sprints, standups, retros. Presumes a team of several. Overkill for a solo project.

Four increments planned, four delivered: 1. **Increment 1** — API Gateway + Authentication. The security floor. 2. **Increment 2** — Violation Analysis with Gemini. The core business value. 3. **Increment 3** — Event-driven Notifications. The async half and the pub/sub demonstration. 4. **Increment 4** — Docker + Kubernetes deployment. The operations story.

Chapters 5–8 — The Four Increments

Each increment chapter follows the **same five-subsection structure**: Analysis, Design, Implementation, Testing, Evaluation. When an examiner opens Chapter 7, they find exactly the same shape as Chapter 5. This is deliberate — predictability helps the reader focus on content.

Chapter 5 — Increment 1: API Gateway + Authentication

Goal. Deliver the *security perimeter*: the single entry point (Gateway) and the identity back-end (Auth). After this increment, a user can register, log in, get tokens, and have those tokens verified on every request — even though no real business endpoint exists yet.

§5.1 — Analysis

In scope: FR2 (Authentication) and the gateway pipeline as a prerequisite for everything else. Dominant non-functional: NFR3 (Security) — bcrypt, short-lived access tokens, refresh rotation, verify at the gateway not at services.

Use case (Figure 5.1): citizen registers with email+password → logs in → gets access+refresh tokens → presents the access token on every subsequent request.

§5.2 — Design

Two patterns: API Gateway + Database-per-Service.

- **Gateway** is Express.js. It exposes public endpoints (`/auth/register`, `/auth/login`, `/violations/*`) and has an `authenticate` middleware that runs before every protected endpoint. The middleware extracts the bearer token, POSTs to the Auth Service's `/auth/verify`, attaches the decoded payload to `req.user`, and calls `next()`. On failure: immediate HTTP 401.
- **Auth Service** owns `snappark_auth` database with two tables: `users(id, email, password_hash, created_at, updated_at)` and `refresh_tokens(id, user_id, token, expires_at, created_at)`. UUID primary keys via `gen_random_uuid()`. Indexes on lookup fields.

Figure 5.2 sequence diagram: Client → Gateway → Auth → (success) → target service. Figure 5.3 high-level design.

§5.3 — Implementation

- **API Gateway** = `services/api-gateway/src/index.js` (289 lines). Uses: `helmet` (HTTP security headers), `cors`, `morgan` (logs), `express-rate-limit` (100 req / 15 min), `axios`, `multer` (to forward multipart uploads unchanged).
- **Listing 5.1** shows the `authenticate` middleware — 20 lines that do the whole token-forwarding dance.
- **Auth Service** = `services/authentication-service/src/index.js` (311 lines) + `helpers.js` + `db.js`. Public endpoints:
  - `POST /auth/register` — validate → check duplicate → bcrypt at cost 10 → persist → issue 15-min access + 7-day refresh tokens → persist refresh.
  - `POST /auth/login` — validate → lookup → bcrypt compare → issue fresh tokens → persist refresh.
  - `POST /auth/verify` — verify access token signature + expiry → return decoded payload.
  - `POST /auth/refresh` — verify refresh token is still in DB and not expired → revoke it (rotation) → issue new pair.
  - `POST /auth/logout` — revoke user's refresh tokens.
- **Listing 5.2** shows the two JWT helper functions — `createAccessToken` and `createRefreshToken`. Both are HS256-signed JWTs. Access and refresh tokens use **two independent secrets**, so a leaked access secret doesn't invalidate refresh tokens and vice-versa.

§5.4 — Testing

Auth has the most thorough test suite of the project: - **Unit tests** (`tests/unit/helpers.test.js`) — email/password validators, token generators, bearer-token extractor. - **Integration tests** (`tests/integration/auth-flow.test.js`) — Supertest hitting the HTTP surface against a throwaway PostgreSQL rebuilt before each run.

The chapter lists **15 passing tests** covering register (201/400/409), login (200/401), verify (200/401), refresh (rotation + reuse rejection).

Manual testing of the gateway with curl/Postman: a request to `/violations/analyze` without a token → 401 without ever contacting the downstream service.

§5.5 — Evaluation

FR2 fully satisfied. NFR3 materially satisfied. Small (~600 LOC across both services). Gap: the gateway itself has no automated tests — flagged as future work in §10.

Chapter 6 — Increment 2: Violation Analysis with Gemini

Goal. The **core business value**: user uploads image → system validates → asks Gemini → returns structured, human-readable verdict. Most technically dense chapter of the dissertation.

§6.1 — Analysis

In scope: FR1 (Upload), FR3 (Validation), FR4 (Case Retention), FR7 (Cleanup), FR8 (Multiple Images). Dominant non-functionals: NFR4 (< 5 s end-to-end), NFR8 (service owns its DB exclusively), NFR6 (Auditability — events emitted).

§6.2 — Design

Three pipelines wired inside the Violation Analysis Service:

1. **Pre-flight validation.** An `imageValidator` uses the Sharp library to check:
   - **Resolution** ≥ 200×200,
   - **Brightness** mean in [30, 245] on 0–255,
   - **Sharpness** Laplacian variance ≥ 100. Failures → HTTP 422 with a human-readable reason explaining what to fix.
2. **LLM reasoning.** Images that pass validation are sent to Gemini 1.5 Flash via the `@google/generative-ai` SDK. Prompt is engineered to return strict JSON with four fields: `violationConfirmed`, `violationType`, `confidence`, `explanation`. A defensive parser strips stray markdown fences.
3. **Persistence + event emission.** Verdict and image metadata are persisted to `snappark_case`. A `case.created` event is published on the RabbitMQ `snappark` topic exchange, so the Notification Service (and, later, an audit writer) can react asynchronously.

Figure 6.2 (sequence diagram): Client → Gateway → Analysis → Validator → Gemini → DB → Broker. Figure 6.3: high-level design.

§6.3 — Implementation

Largest service in the project: `src/index.js` alone is 686 lines; total ~1,200 LOC across `index.js`, `db.js`, `gemini.js`, `imageValidator.js`, `cleanup.js`, `rabbitmq.js`.

§6.3.1 — Prompt Engineering

The prompt (Listing 6.1) plays three roles at once: - **Grounds the model** as an expert traffic warden → biases it toward relevant vocabulary. - **Defines an output schema** → parsing is trivial downstream. - **Constrains the model** to only confirm when the violation is actually visible → reduces false positives.

A second prompt (`MULTI_IMAGE_PROMPT`) is used for FR8, explicitly telling the model to use combined evidence from multiple images.

§6.3.2 — Image Quality Validation

Three tests in sequence, short-circuits on first failure: 1. **Resolution** — compare `metadata.width` and `metadata.height` against thresholds. 2. **Brightness** — convert to greyscale, mean of pixel values. 3. **Sharpness** — convolve a Laplacian kernel with the greyscale image, compute variance of the result. Blurry image → low variance (edges are smeared); sharp image → high variance.

Listing 6.2 shows the sharpness code. The kernel is `[0,1,0, 1,-4,1, 0,1,0]`, a standard 3×3 Laplacian. `edgeVariance = stdev ** 2`. If below `BLUR_THRESHOLD`, reject.

Two important properties: - Catches common failure modes (night photos, dirty windscreen, photo of a wall) *before* wasting a Gemini call → saves latency and cost. - Pure synchronous function over a Buffer → trivially unit-testable.

§6.3.3 — Persistence and the Case Model

`snappark_case` contains a `cases` table (final verdict + lifecycle status `pending | completed | reported_to_authority | resolved | cancelled | expired`), plus companion schema for image metadata and raw Gemini JSON so the parsing logic can evolve without losing history. The auxiliary tables are in `databases/schema.md` and populated as the feature surface expands.

§6.3.4 — Cleanup Job and Additional Features

`cleanup.js` implements FR7: every hour, query `cases` for rows in `pending` older than a configurable threshold (24 h default), mark them `cancelled`/`expired` and remove image files from disk. Plus user-facing read endpoints: - `GET /violations/:caseId` - `GET /violations/:caseId/status` - `GET /violations/user/:userId/cases` (pagination + status filter) - `GET /violations/user/:userId/stats`

§6.4 — Testing

Three unit-test modules: - `gemini.test.js` — mocks the SDK, tests parsing (markdown fences, malformed JSON). - `imageValidator.test.js` — synthetic images at various resolutions/brightness/blur; accept/reject assertions. - `cleanup.test.js` — query-and-mark logic against an in-memory DB double.

24 tests passing across the three modules.

End-to-end manual test: 20 real photos of parked vehicles (some legal, some clearly illegal). System correctly classified **17/20**; the three misses all involved non-Latin signage — a known limitation of the underlying model, honestly acknowledged.

§6.5 — Evaluation

Most important increment. FR1, FR3, FR4, FR7, FR8 all satisfied. The Gemini module isolation successfully mitigates R1 as planned. Pre-flight validator keeps end-to-end latency comfortably under the 5 s target (NFR4).

Chapter 7 — Increment 3: Event-Driven Notifications

Goal. The **async half** of SnapPark. Consumes events from RabbitMQ, fans out notifications across multiple channels. This chapter **demonstrates the extensibility claim** — adding a channel = new class + env var.

§7.1 — Analysis

Implements FR5. Supports NFR5 (maintainability), NFR7 (extensibility), NFR2 (availability) by decoupling notification from analysis.

§7.2 — Design

Three logical components: - **Consumer** — pulls messages from RabbitMQ queues. - **Dispatcher** — decides which channels to fire for each event. - **Channel pool** — pluggable senders.

Three queues declared: - `notification-service.case-created` - `notification-service.case-reported` - `notification-service.case-resolved`

Each has a **dead-letter queue** — poison messages are isolated after the first redelivery rather than looping forever. QoS **prefetch = 1** means the broker won't push a new message until the current one is acknowledged — correct when messages have side effects and are processed sequentially.

Channel abstraction: abstract `BaseChannel` with `send({ to, subject, message, metadata })`. Four concrete subclasses: `InAppChannel`, `SmsChannel`, `EmailChannel`, `PushChannel`. An index file instantiates only the channels whose credentials are present in env. User preferences drive which channels a user opts into, plus per-channel destination addresses (`phone`, `email_addr`, `fcm_token`).

§7.3 — Implementation

Listing 7.1 — RabbitMQ consumer. Applies the patterns from the RabbitMQ docs: - Durable exchange + durable queues (survive broker restart). - Explicit binding of queues to routing keys. - Per-queue dead-letter configuration (`x-dead-letter-exchange`, `x-dead-letter-routing-key`). - Reconnection with exponential back-off. - Clean shutdown handling. - `nack(msg, false, !redelivered)` — on first failure, requeue; on second, dead-letter.

Listing 7.2 — Dispatcher: - Builds the event-specific message. - Loads user preferences. - Filters channels: must be enabled by user AND registered at runtime (credentials present) AND have a destination address. - Fans out with `Promise.allSettled` — a failure in one channel does **not** prevent delivery on the others.

The four channels are deliberately simple: - **InAppChannel** — INSERT into `notifications` table; client polls. - **SmsChannel** — Twilio SDK. - **EmailChannel** — Nodemailer + SMTP. - **PushChannel** — Firebase Cloud Messaging Admin SDK.

Each under 100 LOC. Each disable-able by omitting credentials.

§7.4 — Testing

Three unit-test modules, 22 tests passing: - `channels.test.js` — channel abstraction + runtime registration. - `dispatcher.test.js` — fan-out logic, filtering, concurrency, message building. - `db.test.js` — `getNotificationPreferences`, `upsertNotificationPreferences`, `insertDeliveryLog`.

End-to-end manual test: bring up Docker Compose, trigger `case.created`, confirm via the RabbitMQ management console that the message is routed to all three queues, observe the Notification Service consume it and write the in-app row to the DB.

§7.5 — Evaluation

The architectural claim from Chapter 2 **materialises** here: - Analysis is decoupled from notification. Analysis never calls Notification directly. - Stop Notification Service → messages pile up in durable queues → restart → messages drain in order

→ no event lost. - Add a hypothetical `WebhookChannel` or `TelegramChannel` tomorrow → **zero changes** to any existing service.

Chapter 8 — Increment 4: Deployment and Orchestration

Goal. Package the system for both a developer laptop and a real cluster.

§8.1 — Analysis

No new FRs. Operationalises NFR1 (scalability), NFR2 (availability), NFR5 (maintainability — reproducible deployment is a precondition for long-term maintenance).

§8.2 — Design

Development target — Docker Compose on one host: - Three PostgreSQL 15 instances on host ports 5432, 5433, 5434. - RabbitMQ 3 management on 5672 (AMQP) + 15672 (HTTP). - Four services: Gateway 3000, Auth 3001, Analysis 3002, Notification 3004. - pgAdmin for DB inspection.

Service discovery inside Compose works via Docker DNS: `http://authentication-service:3001`.

Production target — Kubernetes manifests in `deployment/kubernetes/`: - `snappark` namespace. - `ConfigMap` (non-sensitive) and `Secret` template (sensitive) for configuration. - `StatefulSet` for each PostgreSQL + RabbitMQ with `PersistentVolumeClaim`. - `Deployment` for each service with liveness+readiness probes. - `HorizontalPodAutoscaler` on the Violation Analysis Service — scale up on load. - `Ingress` exposing the API Gateway via e.g. `api.snappark.example.com`.

§8.3 — Implementation

§8.3.1 — Dockerfiles

Common pattern: - `node:20-alpine` base image. - Non-root `app` user. - `npm ci --omit=dev`. - Health-check curls `/health`. - `CMD ["node", "src/index.js"]`.

Resulting images ~150 MB each — small enough for fast horizontal scaling.

§8.3.2 — docker-compose.yml

Listing 8.1 shows the analysis service block. Key detail: `depends_on` with `condition: service_healthy` on both the DB and RabbitMQ → the service doesn't try to connect before the dependencies are actually ready.

§8.3.3 — Kubernetes Manifests

One YAML per resource family. Listing 8.2 shows the `Deployment`: - 2 replicas. - `envFrom: configMapRef + secretRef` — config/secrets injected as env vars. - `livenessProbe` and `readinessProbe` both HTTP GET `/health:3002`. - Resource requests 100m CPU / 128Mi RAM; limits 500m / 512Mi.

A `HorizontalPodAutoscaler` targets this Deployment and scales **2 → 10 replicas** based on CPU utilisation (the canonical "load" proxy for an I/O-bound service waiting on Gemini).

§8.4 — Testing

Operational testing: - Clean macOS → `docker-compose up -d` → system comes up reproducibly. - End-to-end manual runs: register → analyse → notify. - RabbitMQ management UI + pgAdmin for runtime inspection. - `kubectl apply --dry-run=client` + `kubeval` for manifest validation.

§8.5 — Evaluation

One command deploys the system to either environment. Same container images in both. Only the orchestration differs.

Chapter 9 — Evaluation (in detail)

Chapter 1 promised six objectives. Chapter 9 scores each one against evidence.

§9.1 — Evaluation of Objectives

- **Scalability — MET.** Microservices + per-service DB + async notifications. HPA on Analysis Service concretely demonstrates it.
- **High Availability — SUBSTANTIALLY MET.** Broker decouples analysis from notifications. Health-checks + RabbitMQ reconnection with back-off. *Not fully:* a multi-replica PostgreSQL setup would be required for a production-grade claim.
- **Security — MET.** bcrypt cost 10, 15-min HS256 JWTs, refresh rotation, gateway rejects unauthenticated requests, `helmet` for security headers.
- **Auditability — PARTIAL.** Events are emitted; schema for the append-only event store is designed but the event-writer is not yet wired. This is the single most important future-work item.
- **Extensibility — MET.** New notification channel = new class + env var. New service wanting to react to a case = new subscriber; no changes to existing code.
- **Maintainability — MET.** Each service < 1,500 LOC, independently deployable, independently testable, its own README.

Table 9.1 shows the six-row scorecard. Honest: three "Met", one "Substantial", one "Partial", one "Met" (maintainability) — *not* all-green. Markers reward this honesty.

§9.2 — User Interface

Deliberately out of scope. All interactions via HTTP clients. The API is documented in OpenAPI 3.0 (`docs/api/openapi.yaml`) plus a human-readable `docs/api/README.md`. Any future mobile or web client built against that contract plugs in without back-end changes.

Chapter 10 — Conclusion (in detail)

Opening

Restates the question SnapPark set out to answer (*can microservices + LLM be a viable civic-tech foundation?*) and answers it: **yes**. Single-command deployment. Four services. Synchronous HTTP for the user-facing flow, async RabbitMQ for everything off the critical path.

§10.1 — Project Challenges

Three honest admissions:

1. **Designing against hallucination.** Gemini sometimes returns invalid JSON (stray markdown fence, trailing comma, prose preamble). Defensive parser + prompt-engineering campaign to reinforce "ONLY a JSON object". Even so, a small percentage of responses cannot be parsed. The service returns 502 in that case; the case stays in `pending`; cleanup removes it after 24 h.
2. **Getting RabbitMQ right.** The naïve consumer loses messages on restart. Dead-lettering + durable queues + prefetch are **non-optional**. Arrived at the correct configuration only by carefully reading the RabbitMQ docs + Richardson's pattern catalogue.
3. **Scope discipline.** Kubernetes + pgAdmin + OpenAPI + four services was already at the edge of a solo effort. Resisting the temptation to also build a React client, an analytics dashboard and an audit writer was the single most important project-management call. All three recorded as future work.

§10.2 — Future Work

What would take SnapPark from a dissertation prototype to something closer to production:

- **Audit writer service** — small service consuming every event, appending to `snappark_audit.events`. Completes the event-sourcing story.
- **API Gateway test suite** — biggest known coverage gap.
- **End-to-end tests** — Cypress/Playwright covering register → login → analyse → notify against a fresh Compose stack.
- **Performance testing** — k6 or Artillery driving 100 concurrent VUs, validating the 5-second target of NFR4.
- **Mobile client** — React Native or Flutter against the OpenAPI contract.

- **Admin dashboard** — web app for the local authority, consuming `case.reported` events, map of open violations, warden-resolution workflow.
- **Hybrid on-device / cloud vision** — small local classifier rejects obviously-non-parking images before they consume Gemini budget. Moves part of the pipeline to the far edge of the Cloud Continuum.

§10.3 — Final Words

A personal reflection. The headline insight: **"A microservices architecture is not expensive because of the services; it is expensive because of the glue between the services."** Every piece of glue — the auth middleware, the RabbitMQ DLQ config, the `Promise.allSettled` dispatcher, the K8s readiness probe, the Compose health-check — has to be written, tested, understood. Once in place, the payoff is equally real: a failing Notification Service no longer kills an analysis; a new channel does not require redeploying everything; a surge in reports does not require a surge in authentication throughput.

References

Chapter 11 is a bibliography with **41 entries**: Fowler, Newman, Richardson (the three "fathers" of microservices writing), Evans (DDD), Kubernetes docs, Docker docs, RFC 7519 (JWT), Gemini technical report, bcrypt paper, RabbitMQ reliability guide, and the comparables (FixMyStreet, SeeClickFix). The IEEE-style bracketed citations `[1]`, `[2]`, … used throughout the text resolve here.

Examiner angle. A thin bibliography is a red flag. Forty-plus entries, mixing books + official docs + peer-reviewed articles, is what a BSc dissertation is expected to have.

How the chapters connect

A single throughline ties the dissertation together:

```
Objectives (§1.2)
        │
        ▼
Non-Functional Requirements (§3.3)   ← derived from objectives
        │
        ▼
Architectural choices (§2)            ← justified by requirements
        │
        ▼
Increments 1–4 (§5–8)                 ← implementations of those
choices
        │
        ▼
Evaluation (§9)                       ← each objective re-checked
```

Every claim traces forwards and backwards along this chain. If an examiner picks any row and asks “why?”, the dissertation has the answer one chapter up. If they ask “so what?”, the answer is one chapter down.

Things to remember for the viva

- The **aim** is one sentence (§1.1). Memorise it.
- The **six objectives** are the scoring sheet (§1.2). Be ready to list them.
- The **architecture choice** is microservices, justified by Objectives 1, 2 and 5 (§2.1.4).
- The **four services** are Gateway, Auth, Analysis, Notification. Never five, never three.
- The **three synchronous-vs-async split** rationale: user is waiting → sync; off the critical path → async.
- The **hallucination mitigation** is defensive parsing + prompt-engineering + 502 + cleanup. You do not pretend Gemini is infallible.
- The **extensibility demonstration** is Chapter 7's BaseChannel pattern — cite it whenever an examiner questions Objective 5.
- The **honest admissions** are in §9.1 (Auditability partial, Availability substantial) and §10.2 (future work). Don't retreat from them; they are evidence of maturity.

End of detailed explanation.